CE-2103 · Lección 07

# Algoritmos de Compresión

Representar los mismos datos con menos bits




Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

# ¿Qué aprenderemos hoy?

1. Diferenciar la compresión con pérdida y sin pérdida
2. Construir el árbol y los códigos de Huffman
3. Explicar la entropía y el límite de Shannon
4. Comprimir y descomprimir con LZ77, LZ78 y LZW
5. Comparar los algoritmos según sus características

// introducción

# ¿Qué es comprimir?

1

## Datos originales

Un conjunto de bits: texto, imágenes, audio, video.

2

## Algoritmo

Reduce la cantidad de bits requeridos para representar los datos.

3

## Resultado

Datos comprimidos: ocupan menos espacio en disco y viajan más rápido por la red.

**Dos categorías:** compresión **con pérdida** (lossy) y **sin pérdida** (lossless).

// tipos de compresión

# Con pérdida vs. sin pérdida

## Con pérdida (lossy)

- Elimina información innecesaria
- **Irreversible:** el original no se recupera
- Audio, video e imágenes
- Parámetro de calidad: compresión vs. degradación

vs

## Sin pérdida (lossless)

- Reduce el tamaño sin perder información
- **Reversible:** se recupera el original
- Texto, documentos y bases de datos
- Reemplaza patrones repetidos por códigos más cortos

Ejemplos: **JPEG** y **MP3** (con pérdida) · **ZIP** y **GIF** (sin pérdida).

# Con pérdida: el parámetro de calidad




Un parámetro de calidad equilibra la tasa de compresión y la degradación visible (50 %, 33 %, 5 %).

# ¿Por qué usar compresión con pérdida?





Los formatos de alta fidelidad (raw) son enormes: sin compresión lossy, el streaming de audio y video no sería posible.

// tipos de compresión

# Percepción de la calidad

### Distorsión

Diferencia entre los datos originales y los comprimidos.

### Percepción subjetiva

Depende del observador: un audiófilo tolera menos distorsión que una persona promedio.

### Fisiología humana

JPEG elimina detalles poco visibles; MP3, sonidos poco audibles.

Los algoritmos **lossy** optimizan la calidad percibida según los atributos fisio-psicológicos del ser humano.

// tipos de compresión

# Compresión sin pérdida

## Cómo funciona

- Es reversible: se recupera el original
- Usa estadística para hallar patrones repetidos
- Los reemplaza por códigos más cortos
- Ej.: frecuencia de caracteres o de colores

## Algoritmos comunes

**Huffman** **LZW** **LZ77** **LZ78**

**RLE** **Deflate (ZIP)** **BWT**

*En este capítulo: **Huffman**, **LZ77**, **LZ78** y **LZW**.*

> Es la categoría adecuada para texto, documentos y bases de datos, donde no se puede perder información.

// huffman

# Huffman: códigos de longitud variable

Desarrollado por David A. Huffman en 1952. Es un algoritmo de compresión sin pérdida.

### Longitud variable

Códigos cortos para los símbolos frecuentes y largos para los poco frecuentes.

### Árbol binario

Se construye un árbol que asigna un código a cada símbolo.

### Tabla de conversión

Se guarda en el archivo comprimido: es esencial para descomprimir.

> No todos los símbolos necesitan los 8 bits de ASCII: si una letra aparece mucho, conviene darle un código muy corto.

// huffman

# El algoritmo en 6 pasos

1. Calcular la **frecuencia** de cada símbolo en el archivo.
2. Crear un **nodo hoja** por símbolo y ordenarlos de menor a mayor frecuencia.
3. **Unir** los dos nodos menores en un padre y reordenar la lista.
4. Repetir el paso 3 hasta que quede **un solo nodo**.
5. Recorrer el árbol: **0** a las ramas izquierdas y **1** a las derechas.
6. Crear la **tabla de conversión** y comprimir el archivo.

// huffman

# Ejemplo: frecuencias de los símbolos

**Texto original (15 símbolos)**

```
bcaadddccacacac
```



**Frecuencias**

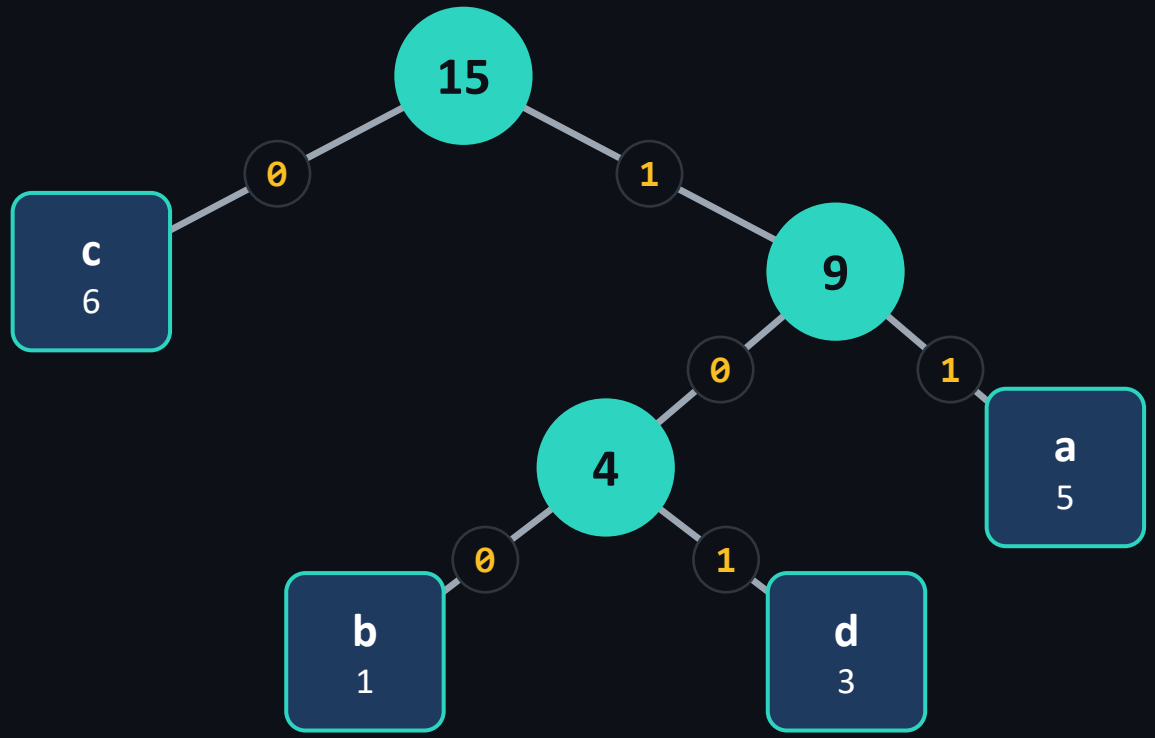
| Símbolo | Frecuencia |
|---|---|
| b | 1 |
| c | 6 |
| a | 5 |
| d | 3 |

En ASCII cada carácter ocupa 8 bits: 8 × 15 = **120 bits** en total.

# Construcción del árbol

| | | |
|---|---|---|
| **Inicio** |  | Hojas ordenadas por frecuencia. Se unen las dos menores: **b** y **d**. |
| **Paso 1** |  | El nodo **4** se inserta según su frecuencia. Se unen **4** y **a**. |
| **Paso 2** |  | El nodo **9** queda al final: tiene la mayor frecuencia. Se unen **c** y **9**. |
| **Paso 3** |  | Queda un solo nodo: la **raíz** (15 = total de símbolos). |

# El árbol y la tabla de conversión

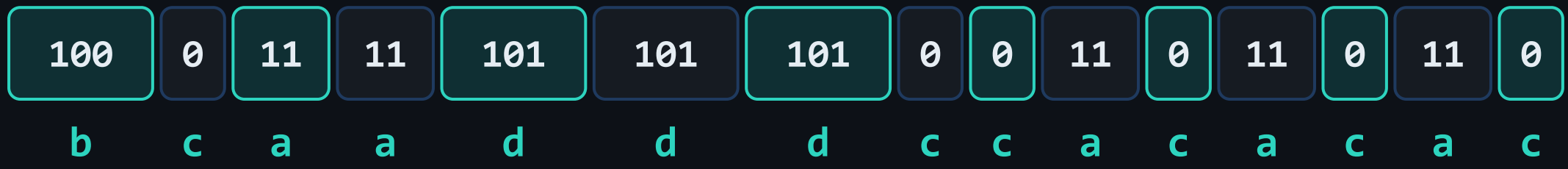



| Símbolo | Frecuencia | Código |
|---|---|---|
| a | 5 | 11 |
| b | 1 | 100 |
| c | 6 | 0 |
| d | 3 | 101 |

**0** a la izquierda y **1** a la derecha. Los símbolos frecuentes quedan cerca de la raíz y reciben códigos más cortos.

# Comprimir el texto

**bcaadddccacacac → 28 bits**







El tamaño de la tabla de conversión también cuenta como parte del archivo comprimido.

# Descomprimir

1

## Leer los bits

La cadena comprimida se trata como un stream de datos.

2

## Recorrer el árbol

Cada bit baja un nivel: 0 a la izquierda, 1 a la derecha.

3

## Emitir el símbolo

Al llegar a una hoja se emite el símbolo y se vuelve a la raíz.

**Ejemplo:** `100` → b, `0` → c, `11` → a, `11` → a, `101` → d, y así hasta el final.

// entropía y límite de shannon

# Entropía de Shannon

## Definición

Para una fuente cuyos símbolos aparecen con probabilidad `p_i`:

$$H = -\Sigma\ p_i \cdot \log2(p_i)$$

*H se mide en bits por símbolo: es la cantidad promedio **mínima** de bits para representar la fuente sin pérdida.*

## Intuición

- Símbolo muy probable: aporta **poca** información
- Símbolo poco probable: aporta **mucha** información
- H es **máxima** si todos son equiprobables y **mínima** si un símbolo domina

// entropía y límite de shannon

# Teorema de codificación de la fuente

## Shannon: límite inferior

Ningún código sin pérdida símbolo a símbolo usa, en promedio, menos de H bits.

$$L \geq H$$

*L = longitud media de código (promedio ponderado por frecuencia).*

vs

## Huffman: óptimo

Óptimo entre los códigos prefijo, pero usa un número entero de bits por símbolo.

$$H \leq L < H + 1$$

*Redundancia = L − H: nunca supera 1 bit por símbolo.*

> Si las probabilidades no son potencias de 2, Huffman queda por encima de H. La **codificación aritmética** y por rangos se acercan más al límite.

# Ejemplo numérico: Huffman alcanza la entropía

| Símbolo | p | log2(p) | -p · log2(p) | Código Huffman | Bits |
|---|---|---|---|---|---|
| **A** | 1/2 = 0.5 | -1 | 0.500 | 0 | 1 |
| **B** | 1/4 = 0.25 | -2 | 0.500 | 10 | 2 |
| **C** | 1/8 = 0.125 | -3 | 0.375 | 110 | 3 |
| **D** | 1/8 = 0.125 | -3 | 0.375 | 111 | 3 |

**H** = 0.5 + 0.5 + 0.375 + 0.375 = **1.75** bits/símbolo

**L** = 0.5·1 + 0.25·2 + 0.125·3 + 0.125·3 = **1.75** bits/símbolo

// lz77

# LZ77: ventana deslizante

Reemplaza secuencias repetidas por referencias anteriores: una tripleta (o, l, c) por paso.

**o**

### Offset

Cuántos caracteres hacia atrás está la secuencia que coincide.

**l**

### Length

Longitud de la secuencia que coincide.

**c**

### Carácter

El siguiente carácter, el que no coincide.

> En cada paso se busca la secuencia **más larga** que coincida con la que inicia en la posición actual y se avanza la ventana.

# LZ77: ejemplo paso a paso




Resultado: `(0, 0, a) (0, 0, b) (2, 2, c) (4, 3, a) (2, 2, a)`. No se necesita ninguna tabla.

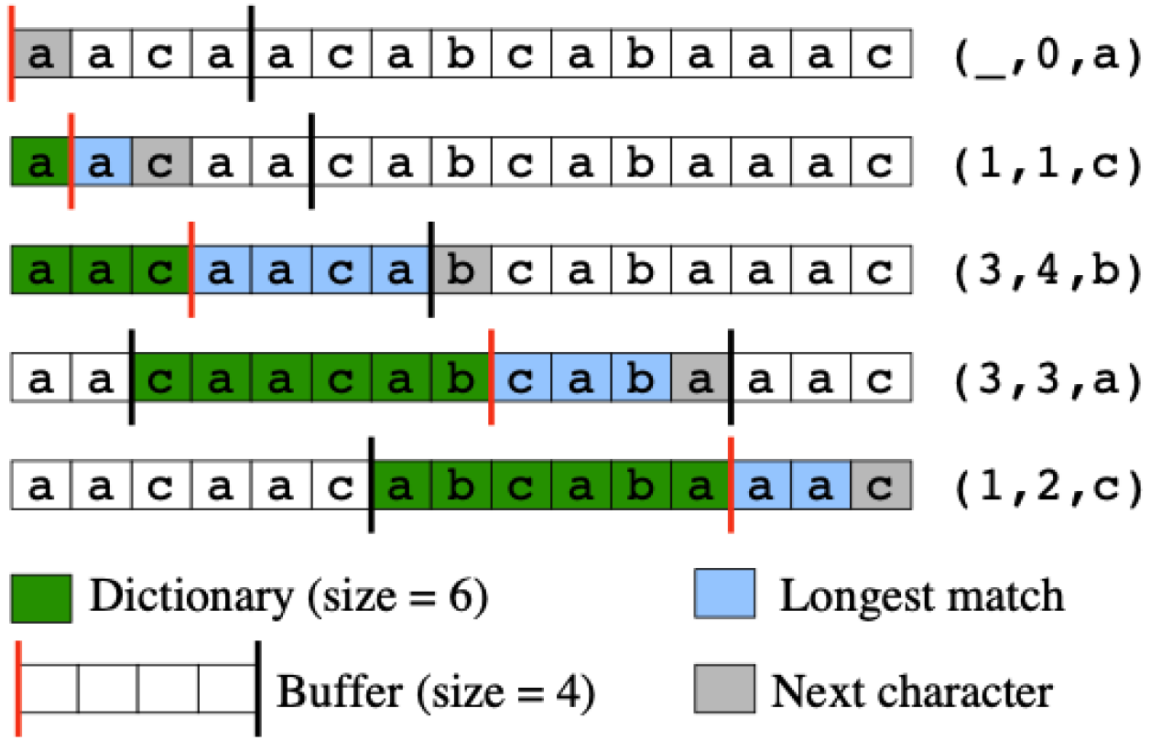
# LZ77: descomprimir

| Tripleta | Buffer resultante |
|---|---|
| **(0, 0, a)** | a |
| **(0, 0, b)** | ab |
| **(2, 2, c)** | ababc |
| **(4, 3, a)** | ababcbaba |
| **(2, 2, a)** | ababcbababaa |

**Regla:** copie `l` caracteres desde `o` posiciones atrás y agregue `c`.

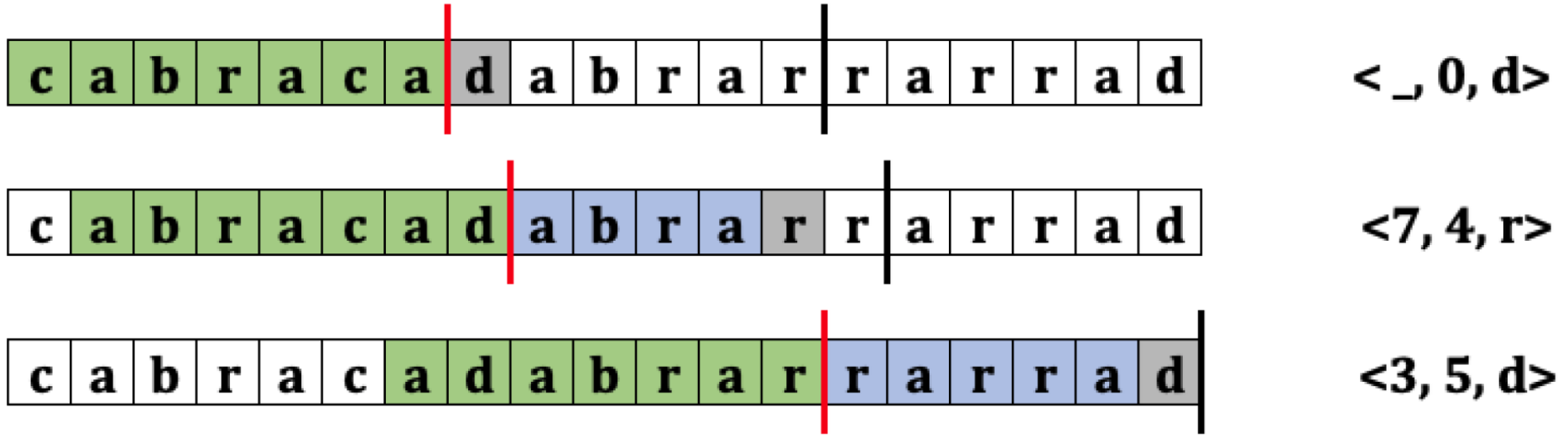
**Ventaja:** no requiere una pasada previa ni una tabla: sirve para comprimir un stream.

# LZ77: ¿por qué 4 coincidencias?




¿Puede explicar por qué la tripleta (3, 4, b) tiene 4 coincidencias?

# LZ77 con diccionario inicial




Con el diccionario inicial «cabraca»: tripletas (_, 0, d), (7, 4, r) y (3, 5, d).

// lz78

# LZ78: diccionario explícito

Lempel y Ziv, 1978: construye un diccionario en lugar de usar una ventana deslizante.

### Diccionario explícito

Se construye de forma incremental con un trie para guardar los prefijos conocidos.

### Sin ventana

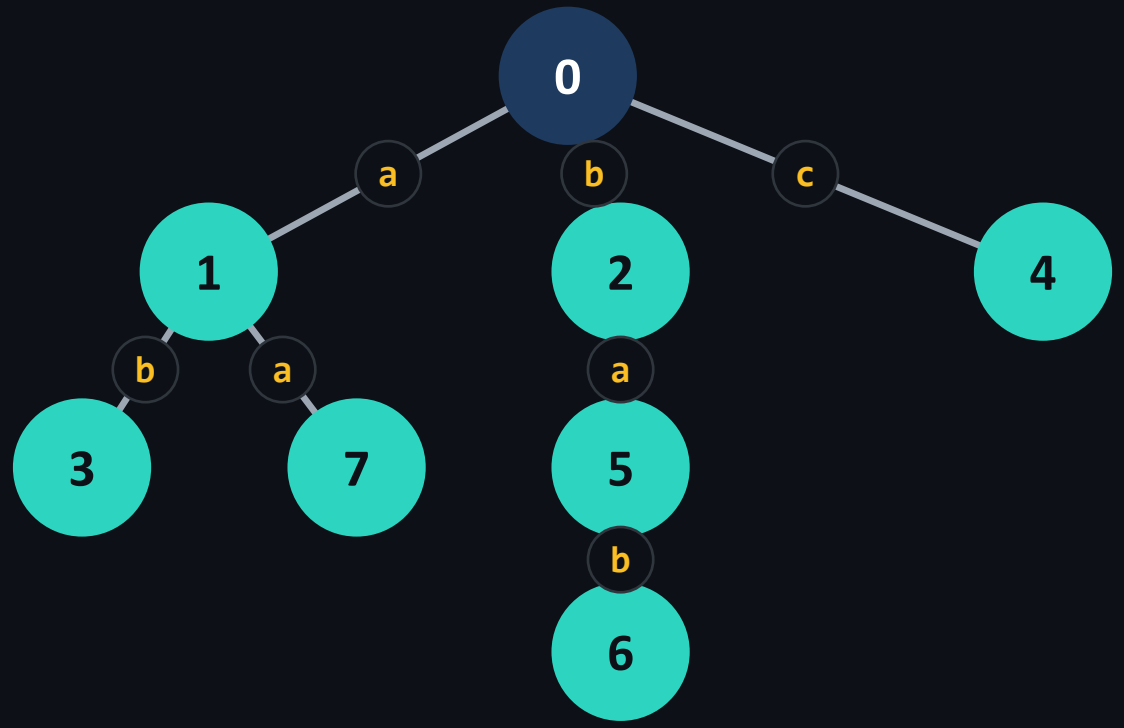
Evita la parametrización que LZ77 requiere para optimizar su desempeño.

### Tupla (o, c)i

o: nodo padre · c: carácter nuevo · i: índice del nodo actual.

> El **trie** es el mismo TDA que vimos en el capítulo de estructuras de datos jerárquicas (CE-1103).

# LZ78: el trie y las tuplas




| Tupla | Cadena |
|---|---|
| (0,a)1 | a |
| (0,b)2 | b |
| (1,b)3 | ab |
| (0,c)4 | c |
| (2,a)5 | ba |
| (5,b)6 | bab |
| (1,a)7 | aa |

Cadena a b a b c b a b a b a a. Descomprimir: desde cada nodo se recorre hasta la raíz. Resultado: ababcbababaa.

// lzw

# LZW: la mejora de LZ78

Terry Welch, 1984. Muy usado en formatos de archivo como GIF y TIFF.

### Tabla de códigos

Los códigos 0 a 255 representan los caracteres ASCII.

### Secuencias nuevas

Identifica secuencias repetidas y les crea un código nuevo.

### Salida

Emite únicamente códigos de la tabla.

> Descomprimir es solo buscar cada código en la tabla y emitir su valor.

# LZW: pseudocódigo

**Compresión**

```
Inicializar la tabla con cadenas de un carácter
P = primer carácter de la entrada
MIENTRAS no sea el fin de la entrada
    C = siguiente carácter
    SI P + C está en la tabla
        P = P + C
    SINO
        emitir el código de P
        agregar P + C a la tabla
        P = C
FIN MIENTRAS
emitir el código de P
```

### P · prefijo

La cadena reconocida hasta ahora.

### C · carácter

El siguiente carácter de la entrada.

# LZW: ejemplo con la cadena abacabaca

| P | C | P + C | ¿Está en la tabla? | Salida | Nuevo código |
|---|---|---|---|---|---|
| **a** | b | ab | No | 97 | ab = 256 |
| **b** | a | ba | No | 98 | ba = 257 |
| **a** | c | ac | No | 97 | ac = 258 |
| **c** | a | ca | No | 99 | ca = 259 |
| **a** | b | ab | Sí | — | — |
| **ab** | a | aba | No | 256 | aba = 260 |
| **a** | c | ac | Sí | — | — |
| **ac** | a | aca | No | 258 | aca = 261 |
| **a** | — | — | Fin de la entrada | 97 | — |

**Salida: 97 98 97 99 256 258 97**. Tabla inicial: a = 97, b = 98, c = 99.

# LZW: descomprimir

### Códigos generados

| 97 | 98 | 97 | 99 | 256 | 258 | 97 |
|---|---|---|---|---|---|---|
| a | b | a | c | ab | ac | a |

### Tabla de códigos

a = 97 · b = 98 · c = 99 · ab = 256 · ba = 257 · ac = 258 · ca = 259 · aba = 260 · aca = 261

> Cada código se busca en la tabla y se emite su valor: `a b a c ab ac a` = `abacabaca`.

// comparación

# Resumen comparativo

| | Año | Estructura | Diccionario o tabla | Característica |
|---|---|---|---|---|
| **Huffman** | 1952 | Árbol binario | Tabla de códigos guardada en el archivo | Códigos de longitud variable según la frecuencia |
| **LZ77** | 1977 | Ventana deslizante | Ninguna: tripletas (o, l, c) | No requiere pasada previa |
| **LZ78** | 1978 | Trie | Diccionario explícito que crece | Sin ventana que parametrizar |
| **LZW** | 1984 | Tabla de códigos | Tabla inicial 0 a 255 que crece | Usado en GIF y TIFF |

Los cuatro son **sin pérdida**: los datos originales se recuperan exactamente.

// cierre

# Resumen

- Comprimir es usar menos bits: **con pérdida** (irreversible) o **sin pérdida** (reversible).
- **Huffman** da códigos más cortos a los símbolos más frecuentes mediante un árbol binario.
- La **entropía** H es el límite: $L \geq H$, y Huffman cumple $H \leq L < H + 1$.
- **LZ77** usa una ventana y tripletas (o, l, c); **LZ78**, un trie; **LZW**, una tabla de códigos.
- **Huffman** necesita las frecuencias; los **LZ** construyen su diccionario sobre la marcha.

// cierre

# Referencias


- https://www.geeksforgeeks.org/what-are-data-compression-techniques/
- https://www.programiz.com/dsa/huffman-coding
- https://hackernoon.com/how-lz77-data-compression-works-yk113te0
- https://hackernoon.com/how-lz78-compression-algorithm-works-x7103tlm
- https://www.geeksforgeeks.org/lzw-lempel-ziv-welch-compression-technique/